rows if no condition is specified). The return values from both calls depend on the adapter but are typically the number of rows affected. An exception is not thrown if the row doesn't exist prior to the call.

```
Order.delete(123)
User.delete([2,3,4,5])
Product.delete_all(["price > ?", @expensive_price])
```

The various `destroy` methods are the second form of row deletion provided by Active Record. These methods all work via Active Record model objects.

The `destroy` instance method deletes from the database the row corresponding to a particular model object. It then freezes the contents of that object, preventing future changes to the attributes.

```
order = Order.find_by(name: "Dave")
order.destroy
# ... order is now frozen
```

There are two class-level destruction methods: `destroy` (which takes an ID or an array of IDs) and `destroy_all` (which takes a condition). Both methods read the corresponding rows in the database table into model objects and call the instance-level `destroy` method of those objects. Neither method returns anything meaningful.

```
Order.destroy_all(["shipped_at < ?", 30.days.ago])
```

Why do we need both the `delete` and `destroy` class methods? The `delete` methods bypass the various Active Record callback and validation functions, while the `destroy` methods ensure that they're all invoked. In general, it's better to use the `destroy` methods if you want to ensure that your database is consistent according to the business rules defined in your model classes.

We covered validation in Chapter 7, *Task B: Validation and Unit Testing*. We cover callbacks next.